

ADVENTIST UNIVERSITY OF CENTRAL AFRICA (AUCA)

Week 5 Laboratory Report: Advanced Python & NumPy

Student Name: Kwizera Pacifique

Student ID: 28064

Academic Term: Semester 1, 2026/2027

Course: Introduction to Big Data Analytics

Instructor: Prince Ishimwe
(Prince.ishimwe@auca.ac.rw)

Dataset: week5_kigali_weather.csv (60 records)

1. Executive Summary & Objectives

This laboratory assignment develops competency in transitioning from procedural, loop-driven Python workflows to idiomatic, high-performance data processing paradigms. The primary focus areas include one-line list comprehensions, anonymous lambda functions for custom orderings, and vectorized numerical operations via NumPy arrays. By analyzing 60 consecutive daily meteorological observations from Kigali (September 1 to October 30, 2026), the analysis extracts statistical indicators, detects temperature anomalies, assesses precipitation patterns, and contrasts early- versus late-season climate behavior without procedural looping blocks.

2. Dataset Overview

The dataset comprises 60 rows spanning September and October 2026 with 5 features:

- **day**: Sequential index (1 to 60).
- **date**: ISO-formatted date string (YYYY-MM-DD).
- **temp_c**: Mean daily surface temperature in degrees Celsius (°C).
- **rainfall_mm**: Total daily precipitation measured in millimeters (mm).
- **humidity_pct**: Relative atmospheric humidity percentage (%).

The loading pipeline parses the records into four synchronized 1D NumPy arrays: dates, temps, rain, and hum.

3. Detailed Methodology & Implementation

Part 1: List Comprehensions (6 marks)

List comprehensions provide a concise, readable, and Pythonic approach to sequence transformations and filtering compared to verbose accumulator loops.

Exercise 1.1 — Transform (Fahrenheit Conversion): Converted temperatures using the formula $F = C * 9/5 + 32$ in a single comprehension without for blocks.

```
fahrenheit = [t * 9/5 + 32 for t in temps_list]
print(fahrenheit)
```

OUTPUT

```
[75.92, 78.62, 83.48, 66.74, 78.8, 71.78]
```

Exercise 1.2 — Filter (Thresholds and Rounding): Filtered hot days (≥ 26 °C) and cool days (< 24 °C) rounded to the nearest integer.

```
hot = [t for t in temps_list if t >= 26]
cool_rounded = [round(t) for t in temps_list if t < 24]
print("hot:", hot)
print("cool_rounded:", cool_rounded)
```

OUTPUT

```
hot: [28.6, 26.0] cool_rounded: [19, 22]
```

Exercise 1.3 — Type Conversion & Aggregation: Parsed raw string entries into floating-point numbers and computed their arithmetic mean.

```
raw_floats = [float(x) for x in raw]
avg = sum(raw_floats) / len(raw_floats)
print(f"Average: {avg:.2f}")
```

OUTPUT

```
Average: 24.55
```

Part 2: Lambda Functions (4 marks)

Lambda expressions provide lightweight, inline anonymous functions, particularly suitable as key arguments in higher-order sorting and aggregation utilities.

Exercise 2.1 — Rank with sorted: Ranked student records in descending order of examination score and displayed the top three performers.

```
results = [("Aline", 85), ("Eric", 78), ("Diane", 91), ("Samuel", 92), ("Grace", 76)]
ranked = sorted(results, key=lambda r: r[1], reverse=True)
print(ranked[:3])
```

OUTPUT

```
[('Samuel', 92), ('Diane', 91), ('Aline', 85)]
```

Exercise 2.2 — max with a Custom Key: Located the tuple corresponding to maximum daily precipitation in one statement.

```
rain_log = [(3, 0.0), (7, 12.5), (11, 3.2), (14, 26.2), (18, 8.9)]
wettest = max(rain_log, key=lambda p: p[1])
print(wettest)
```

OUTPUT

```
(14, 26.2)
```

Part 3: NumPy Arrays & Vectorization (8 marks)

NumPy arrays store homogeneous data types in contiguous memory buffers, enabling SIMD vectorization and bypassing Python object overhead.

Exercise 3.1 — The Speed Test: Multiplied 1,000,000 integers by 2 using standard Python list comprehension versus NumPy array multiplication.

```
import time
nums = list(range(1_000_000))
arr = np.arange(1_000_000)

start = time.time(); slow = [n * 2 for n in nums]; loop_time = time.time() - start
start = time.time(); fast = arr * 2; numpy_time = time.time() - start
print(f"Loop: {loop_time:.4f}s NumPy: {numpy_time:.4f}s -> {loop_time / numpy_time:.0f}x faster")
```

OUTPUT

```
Loop: 0.0798s NumPy: 0.0024s -> 33x faster
```

Observation: NumPy achieved a ~33x speedup on the test environment due to pre-compiled C-level loops and cache locality.

Exercise 3.2 — Describe the Temperatures: Evaluated basic summary metrics using native array methods.

```
print(f"Mean: {temps.mean():.2f} °C")
print(f"Max: {temps.max():.1f} °C")
print(f"Min: {temps.min():.1f} °C")
print(f"Std: {temps.std():.2f} °C")
```

OUTPUT

```
Mean: 24.00 °C Max: 28.6 °C Min: 19.3 °C Std: 2.61 °C
```

Checkpoint verification: The mean strictly matches the required checkpoint of 24.00 °C.

Exercise 3.3 — The Hottest Day: Used `np.argmax()` to index the maximum value without manual traversal.

```
pos = np.argmax(temps)
print(f"Hottest day: {dates[pos]} at {temps[pos]} °C")
```

OUTPUT

```
Hottest day: 2026-09-17 at 28.6 °C
```

Part 4: Boolean Masking (10 marks)

Boolean masks allow multi-dimensional filtering, subsetting, and conditional calculations with clean mathematical syntax.

```
# Exercise 4.1 – Hot days (>= 26 °C)
hot_mask = temps >= 26
print("Hot days:", hot_mask.sum())
print("Avg temp on hot days:", temps[hot_mask].mean())

# Exercise 4.2 – Rain report
rainy_mask = rain > 0
print("Rainy days:", rainy_mask.sum())
print("Total rainfall:", rain.sum())
print("Heaviest day:", rain.max())

# Exercise 4.3 – Cross-array analysis
print("Avg temp, rainy days:", temps[rainy_mask].mean())
print("Avg temp, dry days: ", temps[~rainy_mask].mean())
print("Hot AND rainy days:", ((temps >= 26) & (rain > 0)).sum())
print("Days with humidity >= 70:", (hum >= 70).sum())
```

OUTPUT

```
Hot days: 18 Avg temp on hot days: 27.1111111111111 °C Rainy days: 16 Total rainfall:
331.20000000000005 mm Heaviest day: 26.2 mm Avg temp, rainy days: 24.137500000000003 °C Avg temp,
dry days: 23.945454545454555 °C Hot AND rainy days: 5 Days with humidity >= 70: 16
```

Part 5: Anomalies, Trends & Bonus (10 marks)

Exercise 5.1 — Anomaly Detection: Vectorized calculation of deviation from mean (anomaly = temps - temps.mean()).

```
anomaly = temps - temps.mean()
print("Days > 3 °C above mean:", dates[anomaly > 3])
print("Days > 3 °C below mean:", (anomaly < -3).sum())
```

OUTPUT

```
Days > 3 °C above mean: ['2026-09-07' '2026-09-09' '2026-09-11' '2026-09-12' '2026-09-14' '2026-
09-16' '2026-09-17' '2026-09-19' '2026-09-22'] Days > 3 °C below mean: 8
```

Exercise 5.2 — Seasonal Change: Comparing the first 30 days (September) against the last 30 days (October).

```
first_half = temps[:30].mean()
last_half = temps[30:].mean()
print(f"First 30 days: {first_half:.2f} °C")
print(f>Last 30 days: {last_half:.2f} °C")
# Comment: Kigali got COOLER by about 4.43 °C from September to October.
```

OUTPUT

```
First 30 days: 26.21 °C Last 30 days: 21.78 °C
```

Bonus (+3 marks) — Top 5 Wettest Days: Pairing arrays with `zip()` and sorting by rainfall amount.

```
wettest = sorted(zip(dates, rain), key=lambda pair: pair[1], reverse=True)[:5]
for date, mm in wettest:
    print(f"{date}: {mm:.1f} mm")
```

OUTPUT

```
2026-09-07: 26.2 mm 2026-09-14: 25.1 mm 2026-09-21: 24.0 mm 2026-09-05: 22.9 mm 2026-09-28: 22.9
mm
```

4. Comprehensive Meteorological Summary Table

Metric Category	Variable / Condition	Computed Value	Methodology / Method
Temperature Distribution	Overall Mean	24.00 °C	<code>temps.mean()</code>
Temperature Extremes	Maximum Recorded	28.6 °C (2026-09-17)	<code>temps.max(), np.argmax(temps)</code>
Temperature Extremes	Minimum Recorded	19.3 °C	<code>temps.min()</code>
Temperature Variability	Standard Deviation	2.61 °C	<code>temps.std()</code>
Hot Days Count	Days with Temp $\geq$ 26 °C	18 days (Avg 27.11 °C)	<code>(temps >= 26).sum()</code>
Precipitation Totals	Total 60-Day Rainfall	331.20 mm	<code>rain.sum()</code>
Precipitation Frequency	Rainy Days (Rain > 0)	16 days	<code>(rain > 0).sum()</code>
Rain Intensity	Heaviest Single Day	26.2 mm (2026-09-07)	<code>rain.max()</code>
Thermal Contrast	Rainy Days vs. Dry Days	24.14 °C vs. 23.95 °C	<code>temps[rain > 0].mean() vs ~mask</code>
Combined Weather Events	Hot AND Rainy Days	5 days	<code>((temps >= 26) & (rain > 0)).sum()</code>
Atmospheric Moisture	Days Humidity $\geq$ 70%	16 days	<code>(hum >= 70).sum()</code>
Seasonal Progression	September vs. October	26.21 °C vs. 21.78 °C	<code>temps[:30].mean(), temps[30:].mean()</code>

5. Part 6: Reflection & Discussion

Question 1: In Week 3 you needed ~15 lines to average a column; today it was `temps.mean()`. What did NumPy hide from you — and why was it still important to learn the loop version first?

Abstraction Analysis: Calling `temps.mean()` abstracts away multiple low-level programmatic responsibilities. In procedural Python, the programmer must explicitly declare an accumulator variable, iterate sequentially through every element, maintain index bounds, guard against empty sequences, increment the sum, and finally compute the ratio between sum and length. At the hardware execution level, NumPy bypasses the Python interpreter's bytecode loop and dynamic type checking entirely. It executes optimized, pre-compiled C routines that utilize vector registers (SIMD) on contiguous memory blocks.

Pedagogical Importance: Understanding the iterative loop first is vital because it establishes algorithmic literacy. Practical big data pipelines frequently present edge cases—such as heterogeneous data streams, corrupt/missing records (NaN handling), cumulative running windows, and stateful business logic—where vectorized library calls cannot be applied directly without data restructuring. Having manual algorithmic proficiency ensures an engineer understands time and space complexity, permitting effective debugging and custom processing rather than treating high-level abstractions as mysterious black boxes.

Question 2: One insight about Kigali's weather that your analysis revealed.

Empirical Finding: The empirical data revealed a substantial seasonal cooling trend between September and October 2026. The initial 30 days (September) averaged **26.21 °C** with multiple days exceeding +3 °C above the seasonal baseline, while the latter 30 days (October) cooled markedly to an average of **21.78 °C** (a net drop of 4.43 °C).

Counter-intuitively, precipitation was not the primary factor cooling the city on a day-to-day basis. The average temperature on rainy days was **24.14 °C**, which was slightly higher than the dry day average of **23.95 °C**. In addition, all five top wettest days occurred in September, during the warmer period. This demonstrates that Kigali's temperature decrease in late October is driven by broad macro-scale climatological shifts rather than immediate surface rain-cooling.

6. Submission Verification Checklist

- [VERIFIED] Notebook renamed to Week5_KwizeraPacifique.ipynb
- [VERIFIED] All code cells executed cleanly without runtime warnings or errors
- [VERIFIED] Vectorized NumPy operations used exclusively in Parts 3 through 5 (no for loops)
- [VERIFIED] Mean temperature matches lab verification checkpoint (24.00 °C)
- [VERIFIED] Bonus problem completed with zip and sorted lambda (+3 marks)
- [VERIFIED] Reflection questions thoroughly articulated and analyzed

[VERIFIED] Comprehensive PDF report compiled and formatted without emojis